

DCE/LK/ST3 - Java Coding Assessment

Clinic Appointment & Notification API (Spring Boot + MongoDB)

1. Objective

Build a RESTful API for a clinic to manage Patients, Practitioners (Doctors), Appointments, and Notifications triggered on appointment events.

The solution must demonstrate Spring Boot REST API development, MongoDB persistence, OOP concepts, and at least one well-known design pattern.

2. Tech Requirements (Must Use)

- Java 17+
- Spring Boot 3.x
- MongoDB (local or container)
- Spring Web
- Spring Data MongoDB
- Bean Validation (Jakarta Validation)
- Maven or Gradle
- (Optional) Lombok

3. Business Domain & Entities

3.1 OOP Requirement (Mandatory)

Model people in an OOP way using either:

- Inheritance: Person → Patient, Practitioner
- OR Composition: Patient has Person Details, Practitioner has Person Details

Demonstrate polymorphism in a meaningful way (e.g., notification channel behavior or mapping logic).

3.2 Entities (Minimum Fields)

Patient

- id (Mongo ObjectId as String)
- fullName (required, min 3 chars)
- nationalId (required, unique)
- dateOfBirth (required)
- contact: email (optional, valid), phone (optional)

Practitioner

- id
- fullName (required)

- registrationNo (required, unique)
- specialty (required; e.g., General, Cardiology)

Appointment

- id
- patientId (required)
- practitionerId (required)
- startTime (required, ISO-8601)
- endTime (required, must be after startTime)
- status (BOOKED, CANCELLED, COMPLETED)
- notes (optional)
- createdAt, updatedAt (auto)

4. Core Rules

4.1 Appointment Conflict Rule (Mandatory)

A practitioner cannot have overlapping appointments. Overlap definition:

(newStart < existingEnd) AND (newEnd > existingStart)
(for the same practitioner, where existing status is BOOKED)

4.2 Status Rules

- You can cancel an appointment only if it is currently BOOKED.
- (Optional) Completed appointments cannot be edited — if implemented, document in README.

5. API Requirements (Mandatory Endpoints)

5.1 Patients

1. POST /api/patients — Create Patient
2. GET /api/patients/{id} — Get Patient by Id
3. GET /api/patients?nationalId=... OR ?name=... — Search Patients

5.2 Practitioners

4. POST /api/practitioners — Create Practitioner
5. GET /api/practitioners/{id} — Get Practitioner by Id
6. GET /api/practitioners — List Practitioners

5.3 Appointments

7. POST /api/appointments — Create Appointment (validate conflict rule)
8. GET /api/appointments/{id} — Get Appointment by Id

9. GET /api/appointments?practitionerId=...&date=YYYY-MM-DD&page=0&size=10 — List Appointments (filter + pagination)
10. PATCH /api/appointments/{id}/cancel — Cancel Appointment
11. PATCH /api/appointments/{id} — Update Appointment Notes (Body: { 'notes': '...' })

6. Design Pattern Requirement

Implement at least ONE well-known design pattern and explain it in the README.

- Option A — Strategy Pattern (recommended): NotificationStrategy interface with Email/SMS/Console implementations.
- Option B — Factory Pattern: NotificationStrategyFactory returns strategy based on channel.
- Best: Strategy + Factory together (still simple, shows maturity).

7. Notification Behavior

When an appointment is:

- Created → send 'Appointment booked...'
- Cancelled → send 'Appointment cancelled...'
- Updated (notes) → send 'Appointment updated...'

Notification may be console logging but must be implemented using the selected pattern.

8. Validation & Error Handling

8.1 Validation

- Use Bean Validation annotations for required fields and formats.
- Validate start/end time logic (custom validator or service-layer check).

8.2 Error Response Format

Return consistent error responses like:

```
{
  "timestamp": "2026-04-24T10:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "details": [
    "fullName must be at least 3 characters",
    "endTime must be after startTime"
  ],
  "path": "/api/appointments"
}
```

Handle at least: 400 validation errors, 404 not found, 409 conflicts (appointment overlap).

9. Architecture Expectations (Mandatory)

- Controller (REST)
- Service (business rules)
- Repository (Mongo persistence)
- DTOs for request/response (avoid exposing DB model directly)
- Mapping (manual or MapStruct optional)

10. Testing (Important — Minimum)

- At least 2 unit tests for appointment conflict rule
- At least 1 controller integration test OR service test for happy path appointment creation
- If time is short, prioritize service tests.

11. Deliverables

- 12. Git repository OR zipped project
- 13. README: how to run, endpoints overview, pattern used + why, assumptions & trade-offs
- 14. Postman collection
- 15. (Optional) Swagger/OpenAPI enabled

12. Bonus (Optional)

- Swagger/OpenAPI at /swagger-ui
- Docker composes for MongoDB
- Indexes for uniqueness and faster queries
- More robust filtering/pagination
- Basic authentication (simple API key header)

13. Scoring Rubric (100 Points)

- Core functionality (40): CRUD/listing works + conflict rule correctly implemented
- Code quality & architecture (25): layering, DTO separation, naming/readability, minimal duplication
- OOP + Design Pattern (20): OOP modeling + correct pattern implementation
- Validation + error handling (10): proper 400/404/409 + consistent error format
- Tests + documentation (5): critical tests + README clarity

14. Postman Collection

Save the JSON below as: API.postman_collection.json and import into Postman.